

Transformers - Intro

Advanced NLP M26

Aparajitha



August 3, 2026

Seminar (10%)

- Replace one exam with a presentation in class
- Not more than **16** approved students
- **NOT** present one research paper but thorough presentation on a topic
- Evaluated on topic understanding, coverage, presentation and ability to answer questions
- Details on topics + form opens - post add/drop



Project (50%)

- Propose Custom Projects
- Team size: **3 or 4**
- Random team allotment option available
- Resubmit proposal if rejected
- Utilize TA Office hours to discuss projects feasibility* and scope

* Compute cannot be provided :(



Recap

Word Representations - Evolution

- One Hot Vector
- tf-idf
- Word2Vec, GloVe, fasttext
- BERT
- LLM Embeddings

Context

Low

High

Ambiguity

High

Low

Sparsity

Sparse

Dense

Complex

Low

High



Gradients & Backprop

Gradients

- Gradient (slope) - derivative of a function
- Change in output wrt to change in input

$$f(x) = x^n$$

$$f'(x) = \frac{dx^n}{dx} = nx^{n-1}$$



Gradients

Function with 1 output and n inputs

$$f : \mathbb{R}^n \rightarrow \mathbb{R}, \quad f(x) = f(x_1, x_2, \dots, x_n)$$

$$\nabla f(\mathbf{x}) = \frac{\partial f}{\partial \mathbf{x}} = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$$



Gradients - Jacobian Matrix

Function with m outputs and n inputs

$$f: \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad \mathbf{y} = f(x_1, x_2, \dots, x_n) = (y_1, y_2, \dots, y_m)$$

$$J_f(\mathbf{x}) = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \dots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \dots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$



Hessian Matrix

Second order partial derivative of a scalar-valued function

$$f: \mathbb{R}^n \rightarrow \mathbb{R}, \quad f(\mathbf{x}) = f(x_1, x_2, \dots, x_n)$$

$$H_f(\mathbf{x}) = \nabla^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \dots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \dots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \dots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$



Gradient Descent

- Goal: Minimize objective function $J(\theta)$
- Random Parameter Initialization θ
- Find Slope $\nabla_{\theta} J(\theta)$
- Move

$$\theta = \theta - \eta \nabla_{\theta} J(\theta)$$

Adjust model parameters (weights, biases,) to minimize error

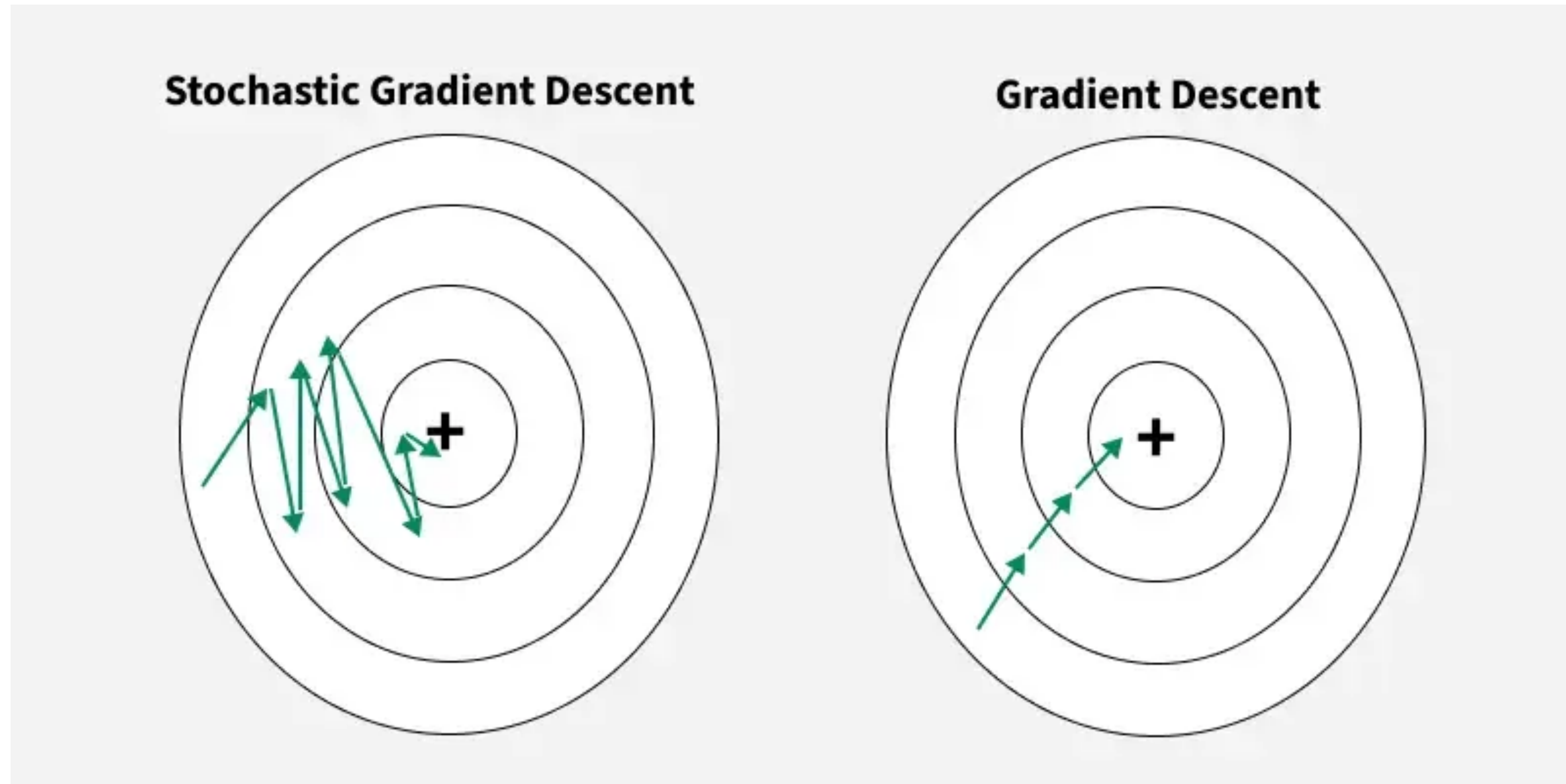
Stochastic Gradient Descent

- Optimization on traditional gradient descent
 - Slow and memory-intensive
- Calculate gradients for each training sample or batch instead of full dataset

$$\theta = \theta - \eta \nabla_{\theta} J(\theta; x_i; y_i)$$



Gradient Descent vs SGD



Learning Rate η

- Step size to descent
- Too small: Slow Convergence
- Too big: Overshoot



Learning Rate Fixes

- Gradient Clipping
- Weight Regularizations
- Batch Normalization



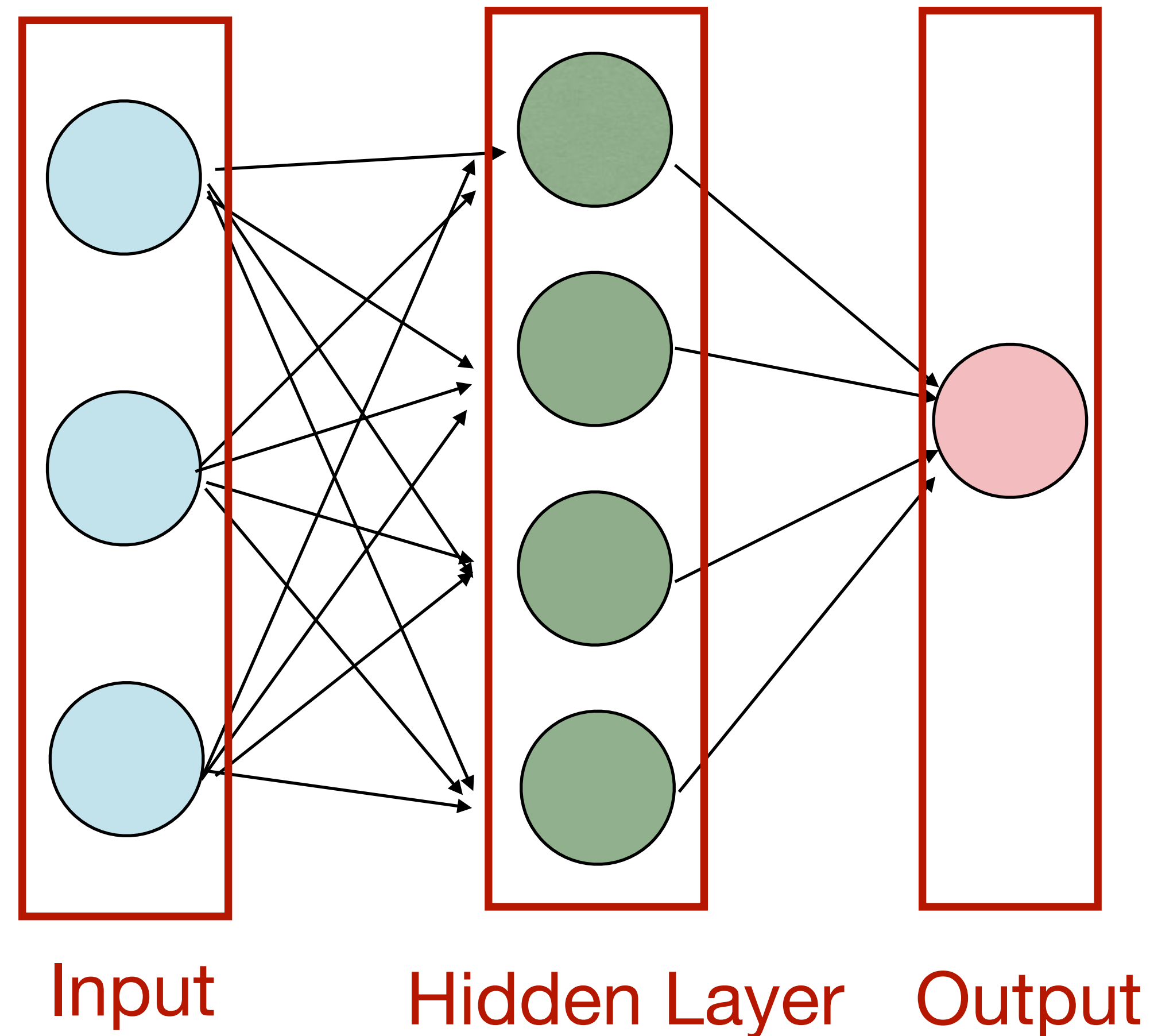
Backpropagation

- Reduce error by adjusting the weights and biases in the network
- Forward pass
- Backward pass



Backpropagation

- Forward pass: Input fed to network to get output

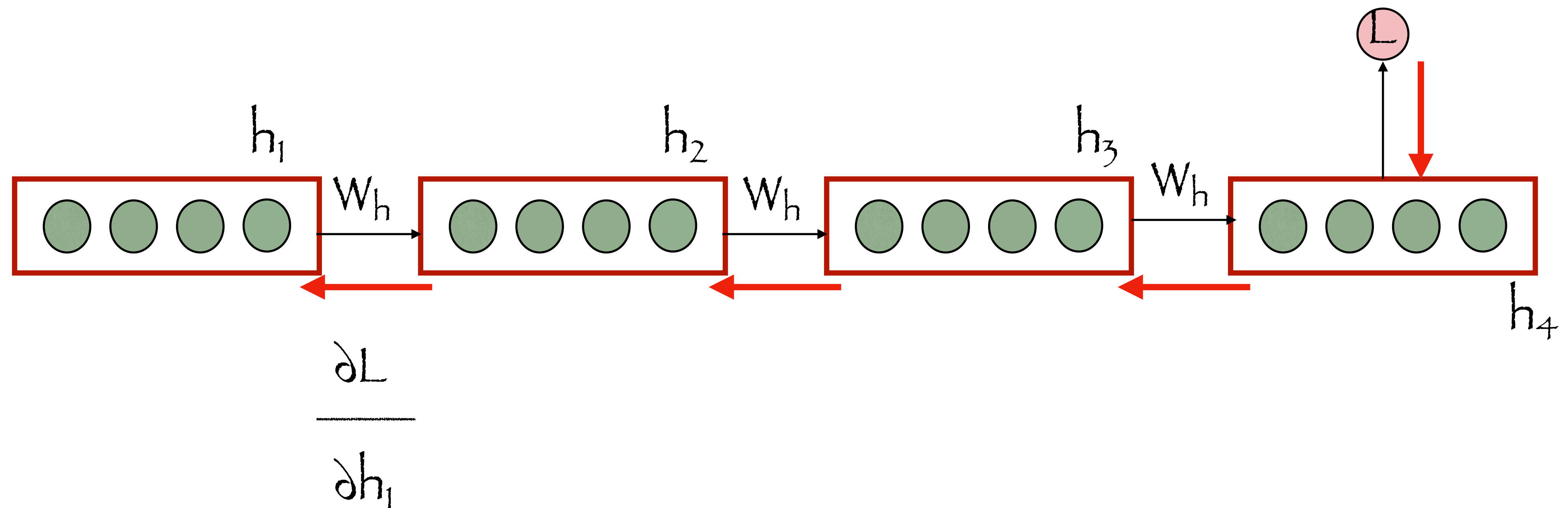


Backpropagation

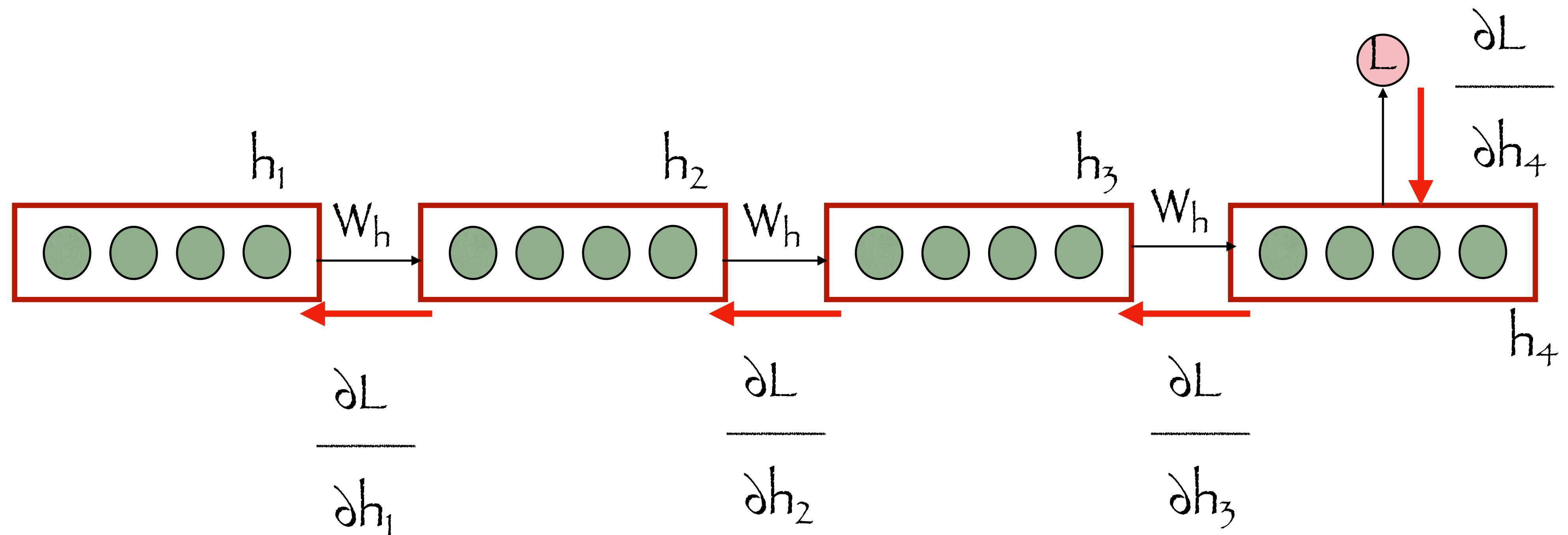
- Backward pass:
 - Error Calculation
 - Propagation of gradients



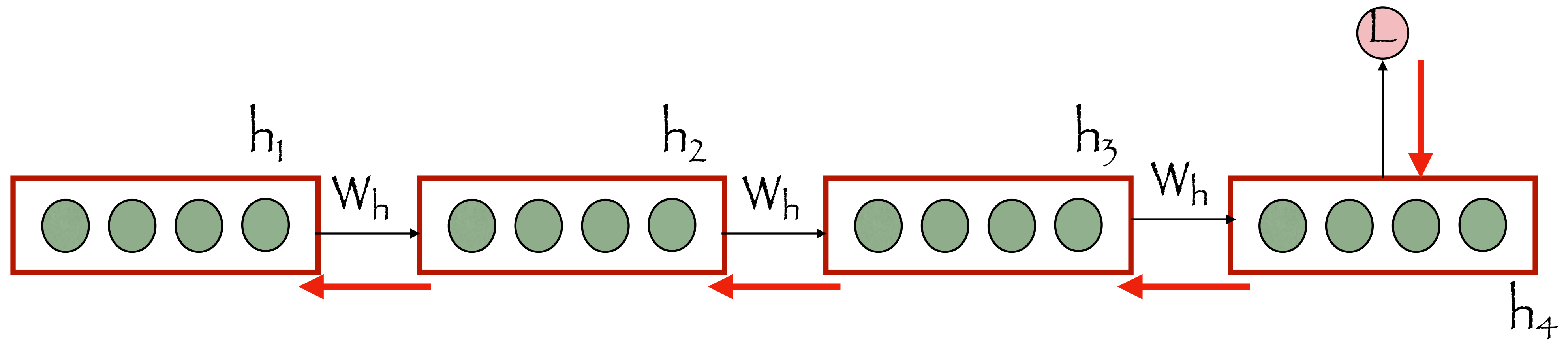
Calculating Gradients



Calculating Gradients



Calculating Gradients

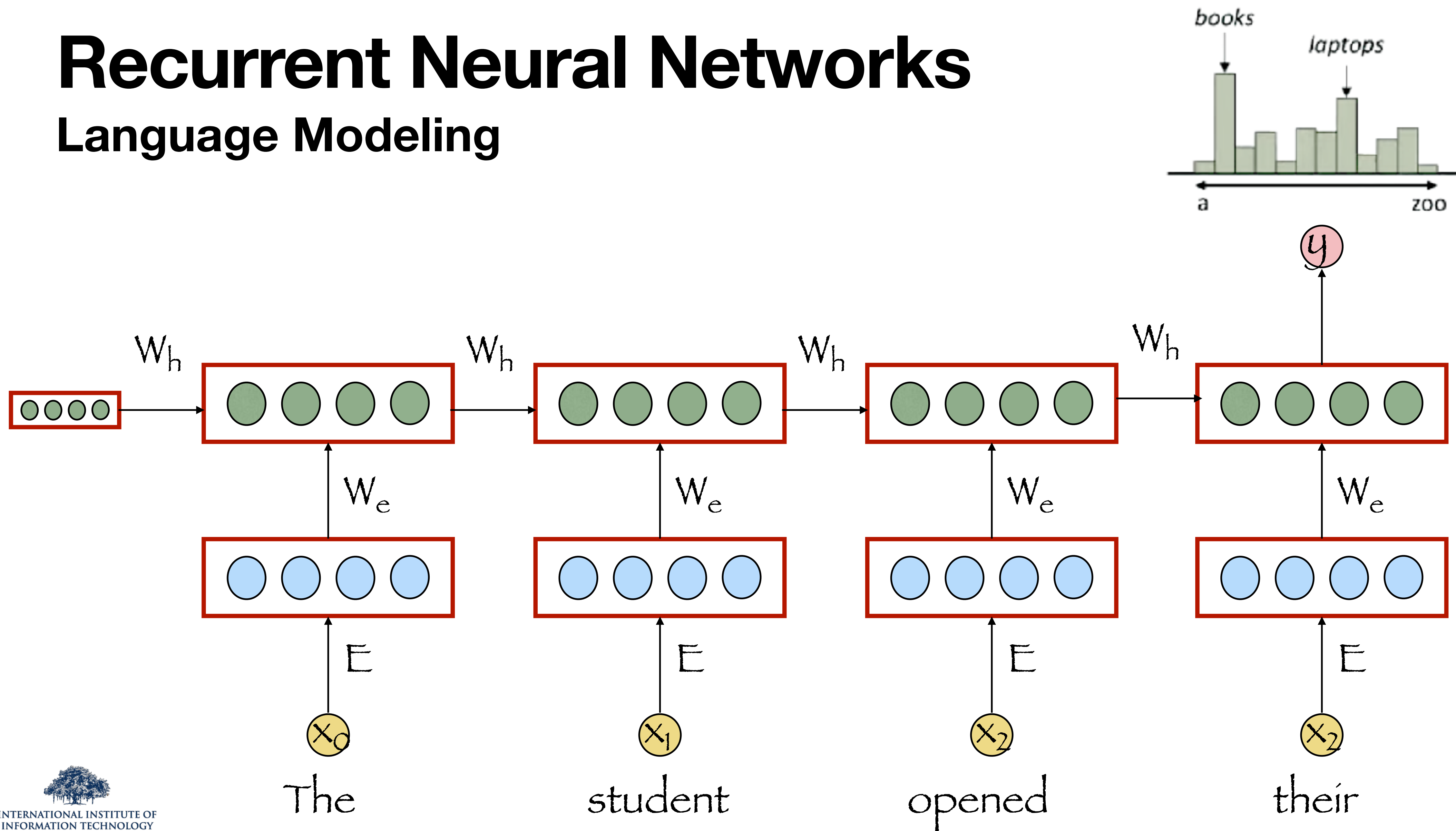


$$\frac{\partial L}{\partial h_1} \approx \frac{\partial h_2}{\partial h_1} \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial h_4}{\partial h_3} \times \frac{\partial L}{\partial h_4}$$

Neural Networks in NLP

Recurrent Neural Networks

Language Modeling



Recurrent Neural Networks

Limitations

- Finite Context
- Long Distance Dependency
- Lack of parallelizability



Vanishing Gradient

- Gradients < 1 get smaller and smaller
- Gradient signal from far is lost
- Model weights updated with only near relations not long-term



Exploding Gradient

- Gradients $\gg 1$
- Update step is too large
- Rarer than vanishing gradients
- Can be fixed with gradient clipping



Long Dependencies in Language

Relationships in Natural Language

- Anaphora
- Co-reference
- Causal
- Temporal
- Discourse Structures



Anaphora - Pronoun

I read the latest bestseller, and I didn't enjoy **it** as much.



Co-reference



John told **Sally** that **she** should come watch him play the violin.



“I didn’t vote for **X** because **they** aligned most with *my* values”, Y said.



Causal Relation

Since you suggested it, I now have to deal with it.



Temporal Relation

Since you suggested it, we are now working on it.



Long Distance Relation

When she tried to print her tickets, she found that the printer was out of ink. She went to a stationary store and purchased some ink. She felt that the ink was very expensive. After refilling the cartridge, she could finally print her _____.



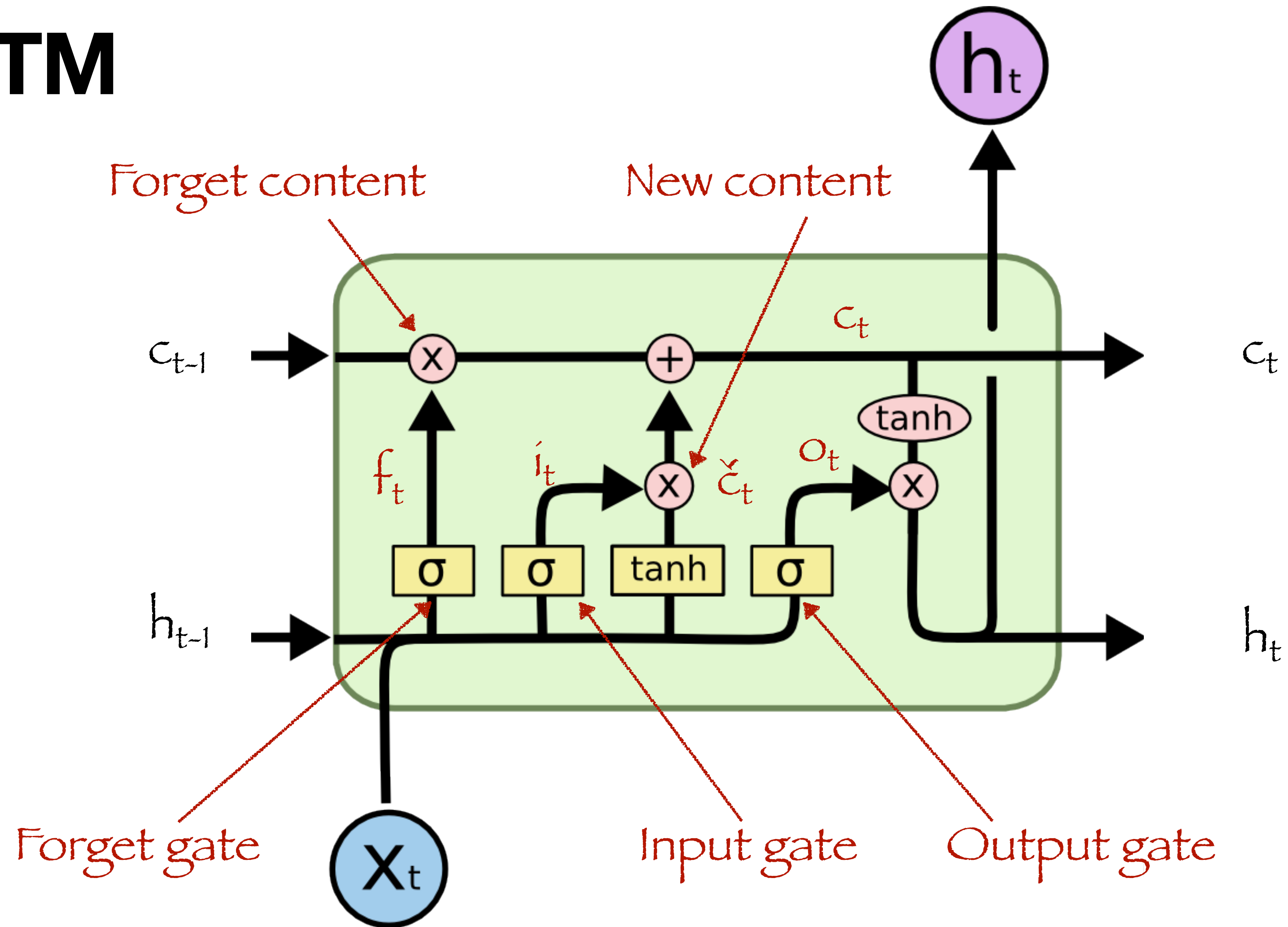
Long Distance Relation

I grew up in Spain and then moved to the UK for my undergrad. After completing my undergrad I took a gap year and left for South America. I was hiking in the Patagonia and then went to Peru. While there I had no trouble speaking with the locals since I am fluent in

_____.



LSTM



LSTM

Advantages over RNN

- Vanishing Gradients - Solved
- Long distance dependency - Partially solved
 - Not solved for very long distances



LSTM

Limitations

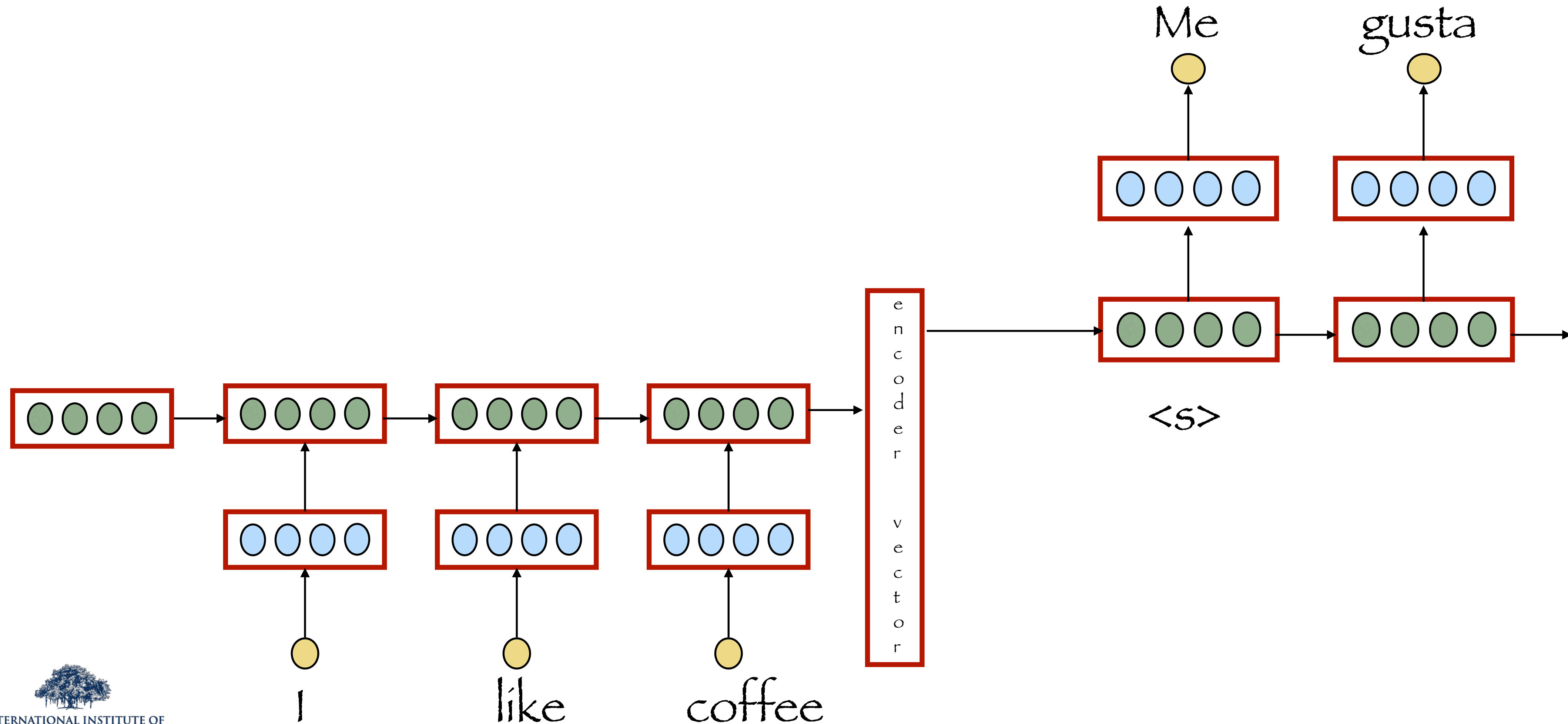
- Computationally Complex
- Slow
- Limited long distance dependency
- Transfer Learning not possible



Attention

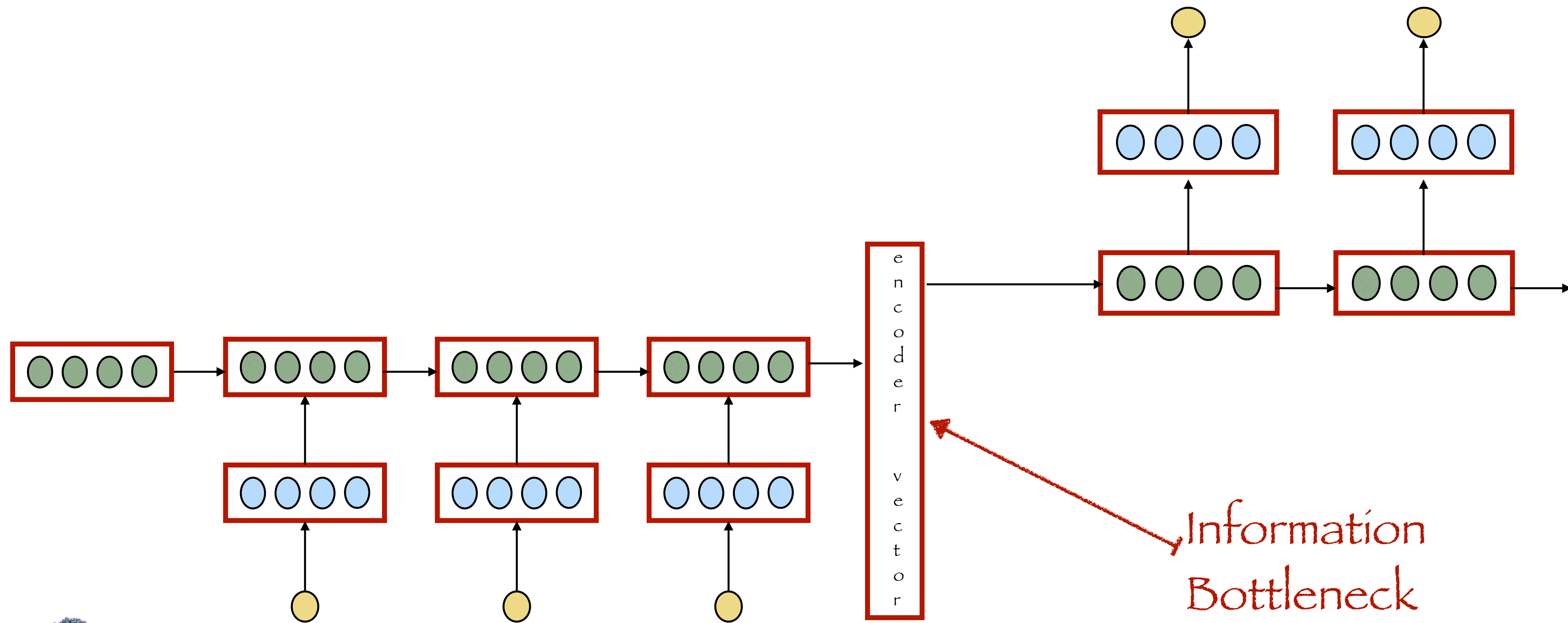
Seq2Seq - Encoder Decoder

Machine Translation



Encoder Decoder

Limitation

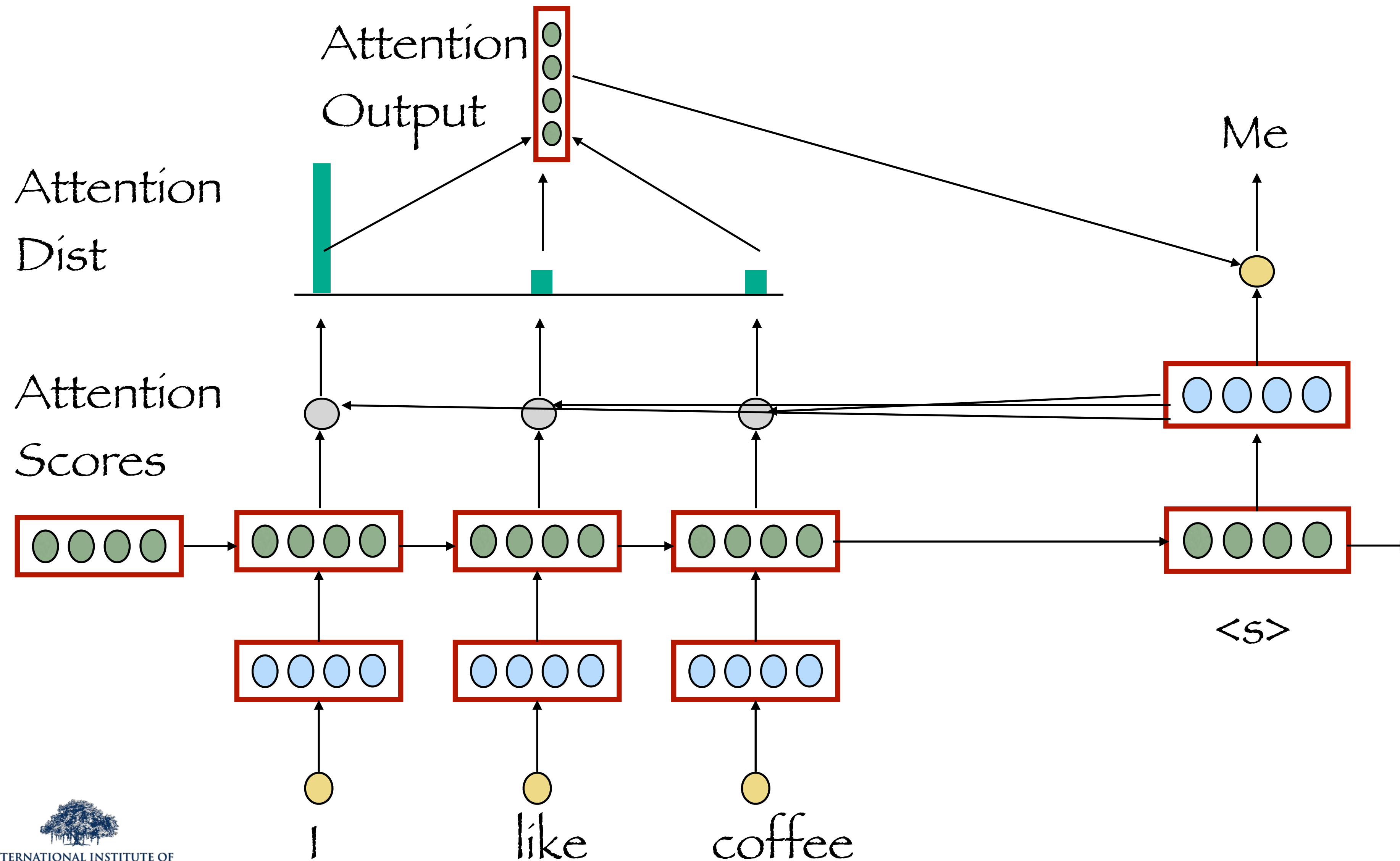


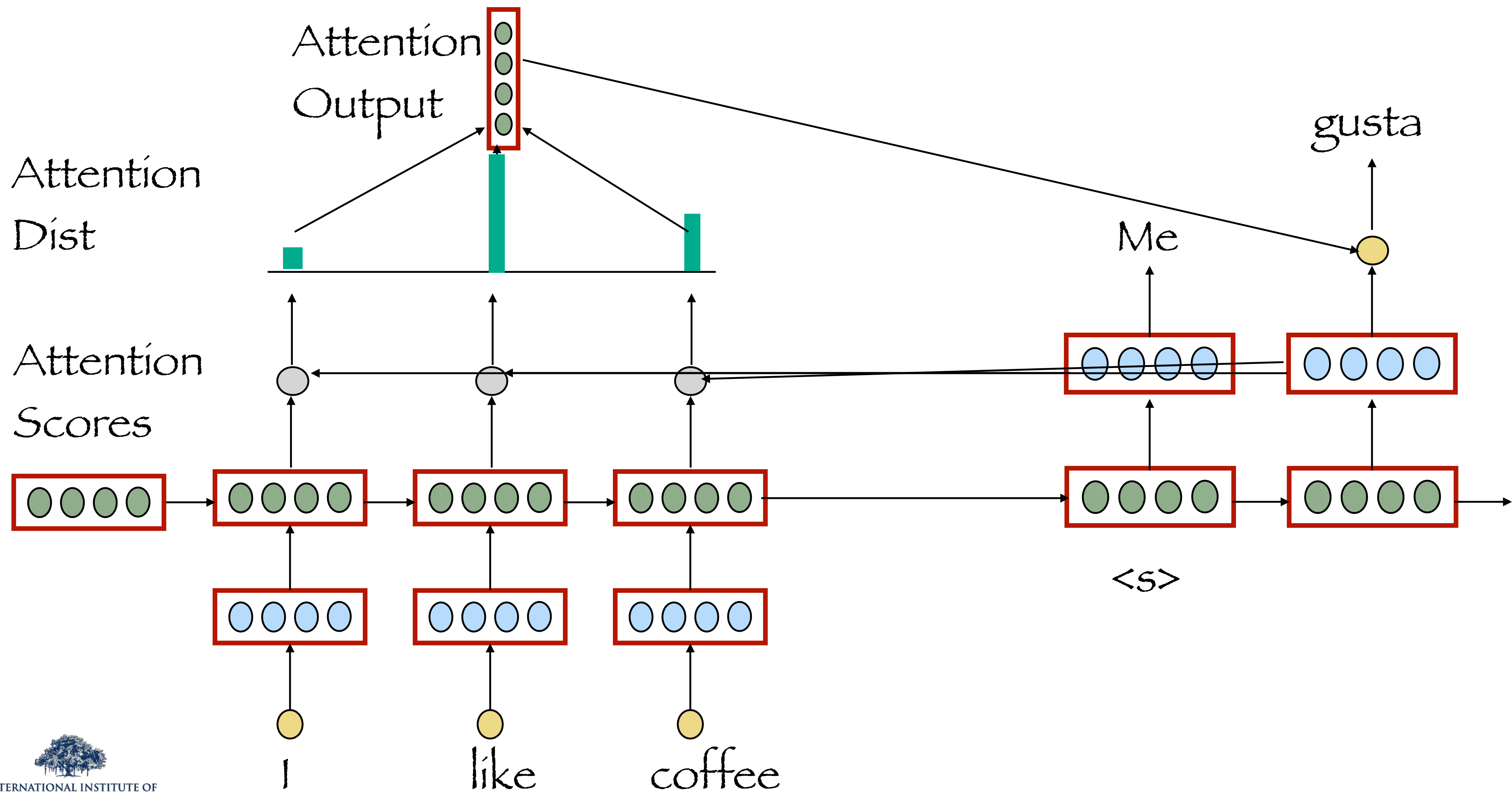
Attention

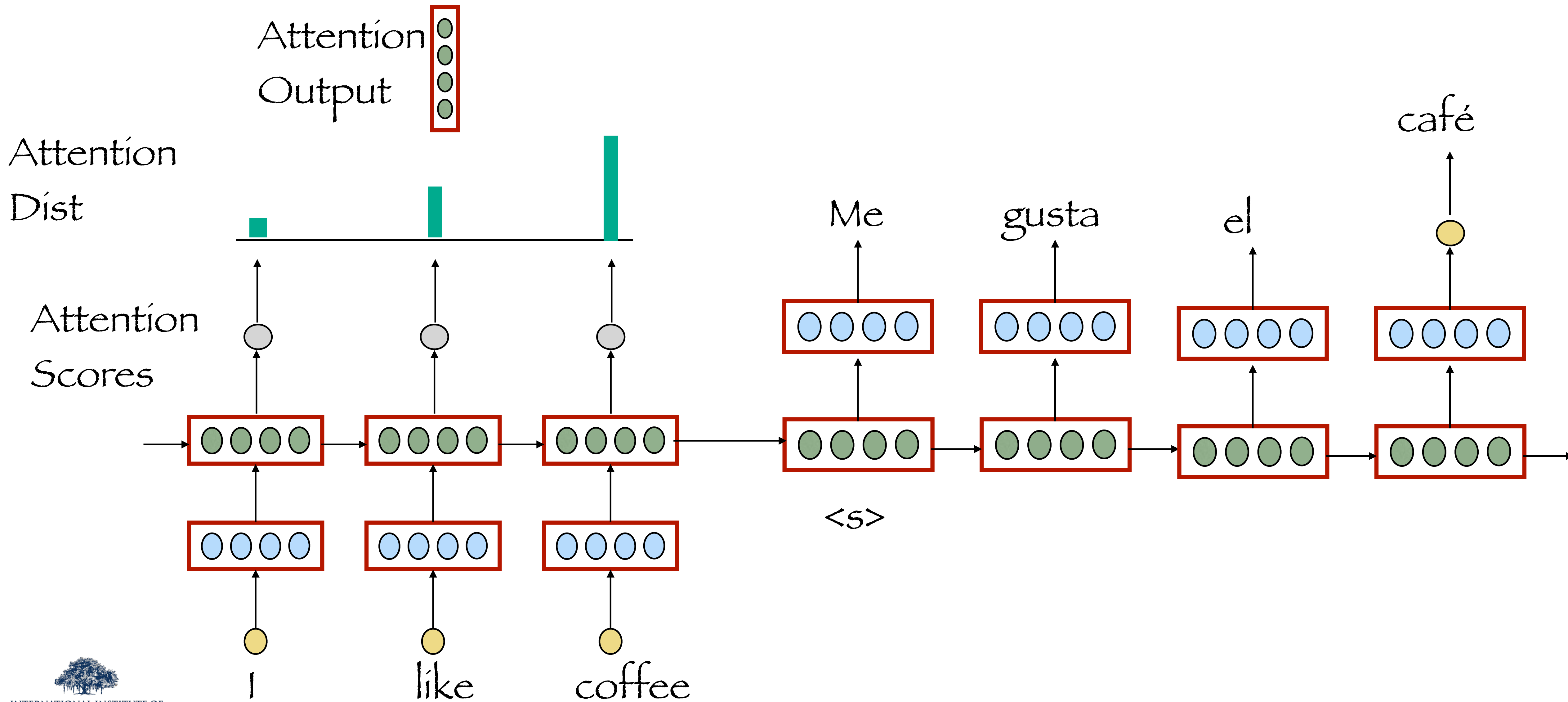
- More information from sentence before translating
- Each decoder step uses direct connection to encoder



Attention Machine Translation







Attention

Advantages

- No Information bottleneck
 - No one hidden state to capture all information about source
- Each decoder step uses direct connection to encoder
- More “human-like” process
- Attention Distribution gives some visualization (interpretability)
- No explicit alignment

Attention

Issues - RNN/LSTM

- Long distance dependency
- Sequential Processing



Transformer

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

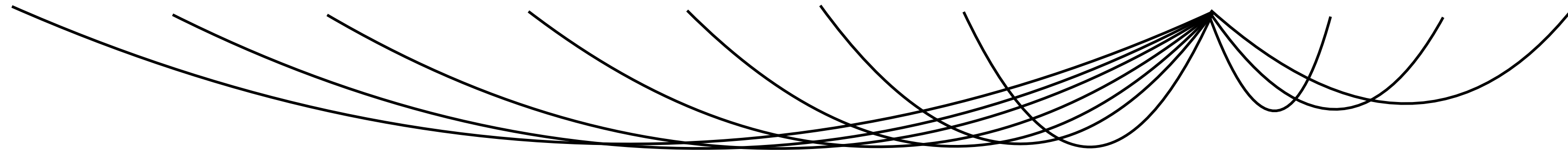
Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Self Attention

Self Attention

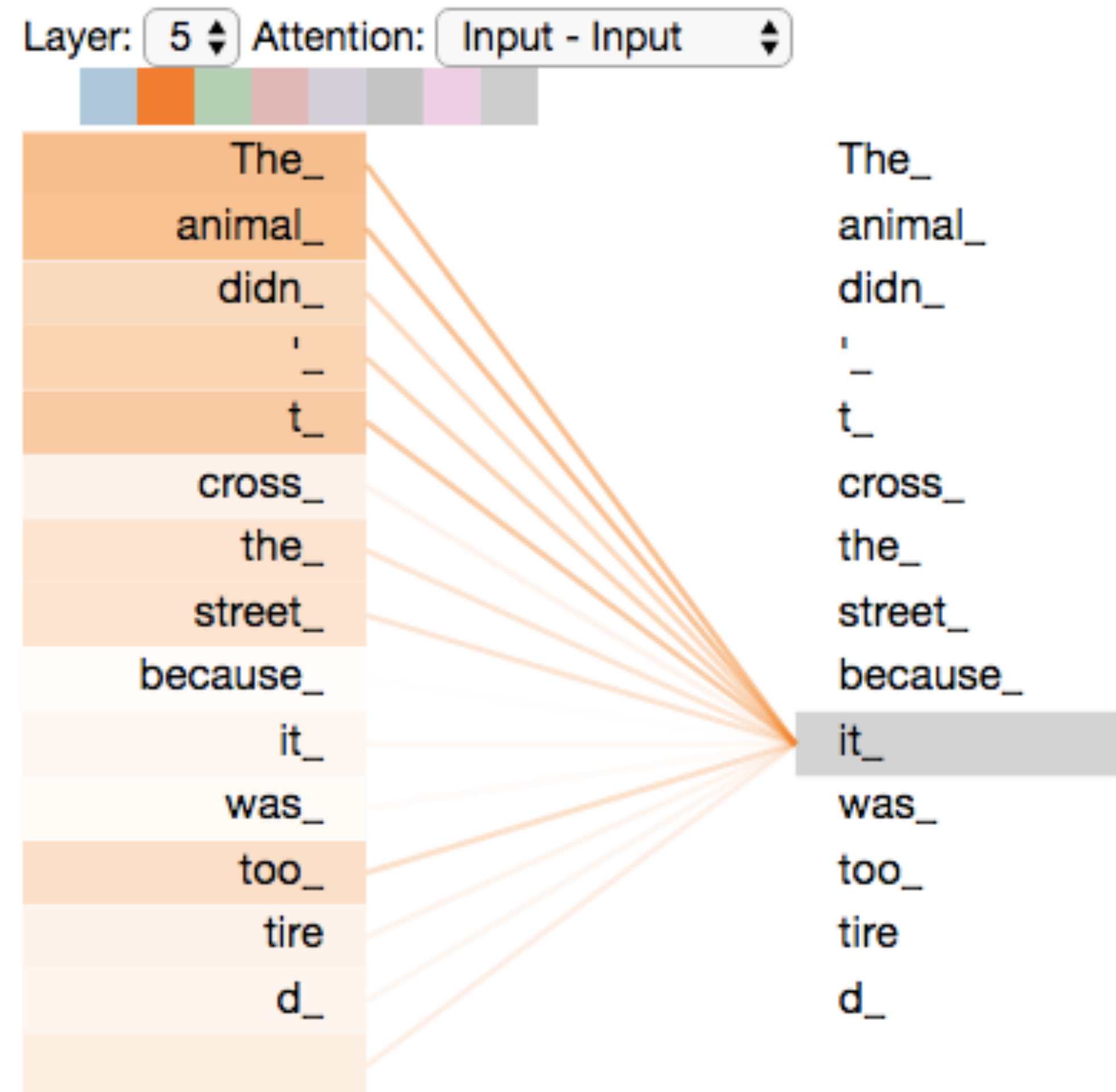
Basics

The animal didn't cross the street because it was too tired



Self Attention

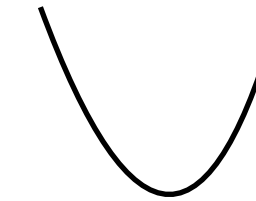
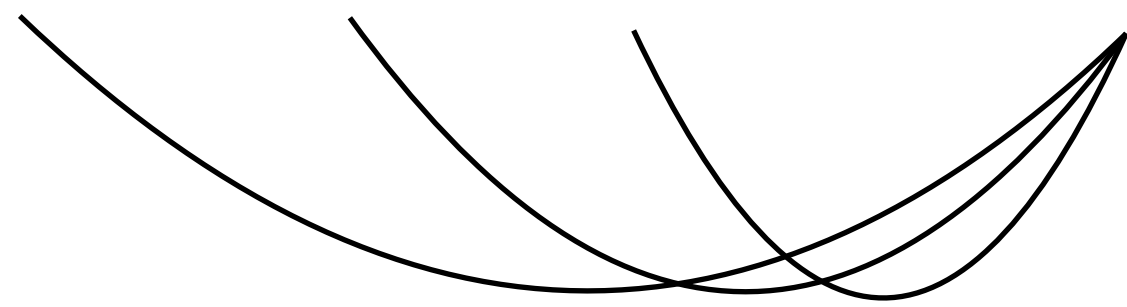
Basics



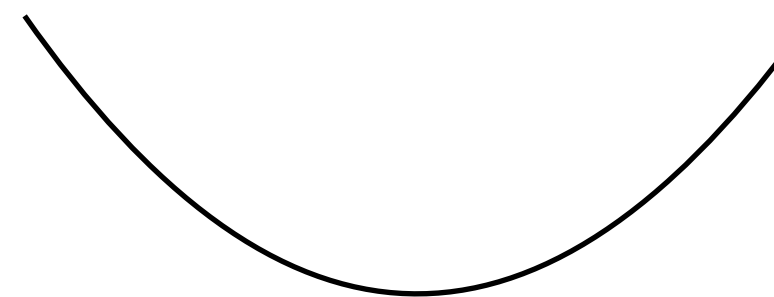
Self Attention

Embedding

The cute fluffy brown cat is lazing over the bright shelf in the corner



The cute fluffy brown cat is lazing over the bright shelf in the corner



Self Attention

Attention Matrix

	A	brown	cat	lazes	on	the	shelf
A							
brown							
cat							
lazes							
on							
the							
shelf							

Self Attention Calculation

$$\textit{Attention}(Q, K, V) = \textit{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$



Self Attention

Embedding

A	brown	cat	lazes	on	the	shelf
E ₁	E ₂	E ₃	E ₄	E ₅	E ₆	E ₇

Self Attention

K, Q, V

A	brown	cat	lazes	on	the	shelf
---	-------	-----	-------	----	-----	-------

k_1	k_2	k_3	k_4	k_5	k_6	k_7
-------	-------	-------	-------	-------	-------	-------

q_1	q_2	q_3	q_4	q_5	q_6	q_7
-------	-------	-------	-------	-------	-------	-------

v_1	v_2	v_3	v_4	v_5	v_6	v_7
-------	-------	-------	-------	-------	-------	-------

Self Attention

K, Q, V

Transform word embeddings with Weight Matrices

A	brown	cat	lazes	on	the	shelf
---	-------	-----	-------	----	-----	-------

$E_i W_K$

k_1	k_2	k_3	k_4	k_5	k_6	k_7
-------	-------	-------	-------	-------	-------	-------

$E_i W_Q$

q_1	q_2	q_3	q_4	q_5	q_6	q_7
-------	-------	-------	-------	-------	-------	-------

$E_i W_V$

v_1	v_2	v_3	v_4	v_5	v_6	v_7
-------	-------	-------	-------	-------	-------	-------

Self Attention

Attention Matrix

$$\frac{QK^T}{\sqrt{d_k}}$$

		q ₁	q ₂	q ₃	q ₄	q ₅	q ₆	q ₇
		A	brown	cat	lazes	on	the	shelf
k ₁	A							
k ₂	brown							
k ₃	cat							
k ₄	lazes							
k ₅	on							
k ₆	the							
k ₇	shelf							

Self Attention

Attention Matrix

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

		q ₁	q ₂	q ₃	q ₄	q ₅	q ₆	q ₇
		A	brown	cat	lazes	on	the	shelf
k ₁	A			0				
k ₂	brown			0.38				
k ₃	cat			0.48				
k ₄	lazes			0.08				
k ₅	on			0.01				
k ₆	the			0				
k ₇	shelf			0.05				

Self Attention

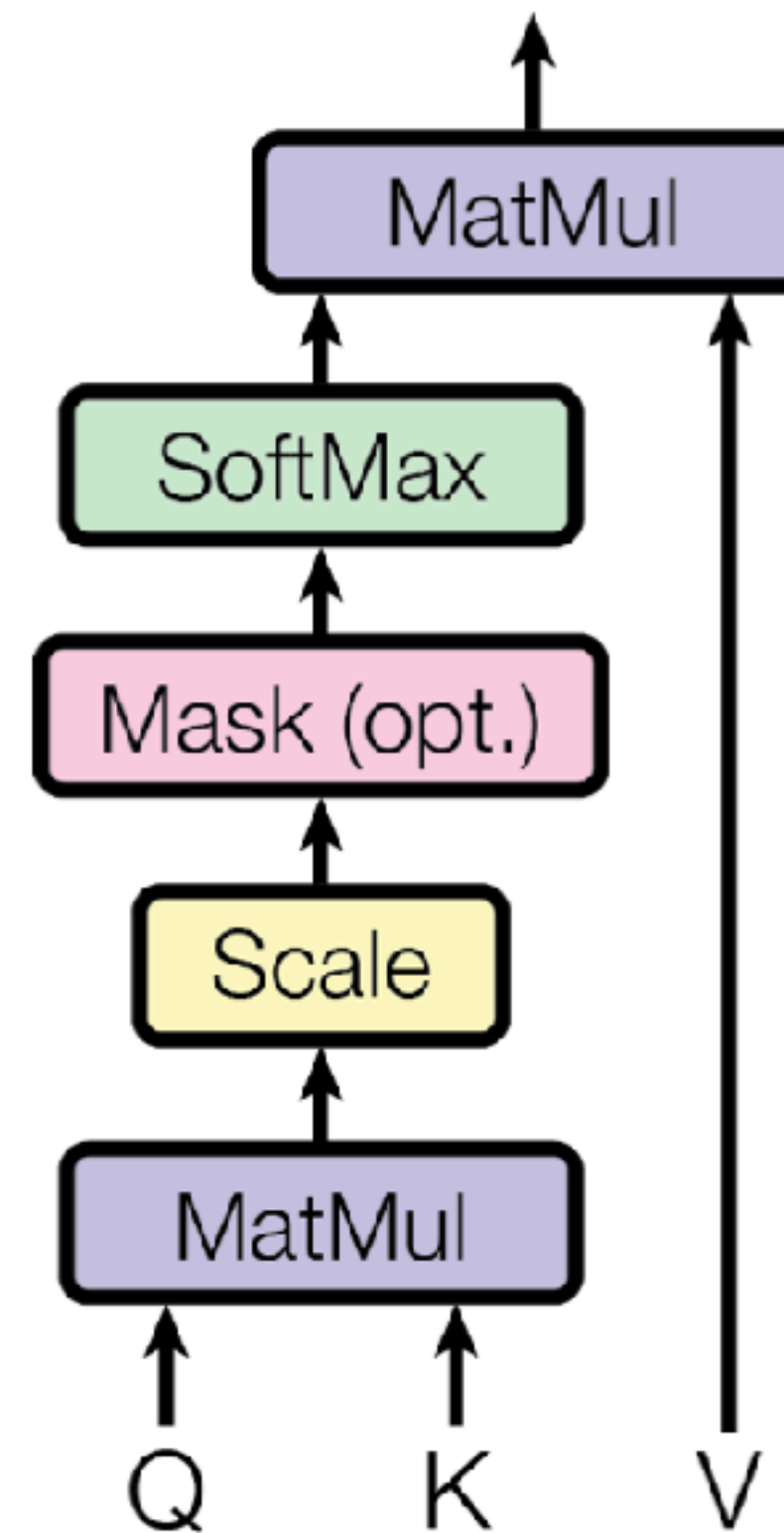
Attention Matrix

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

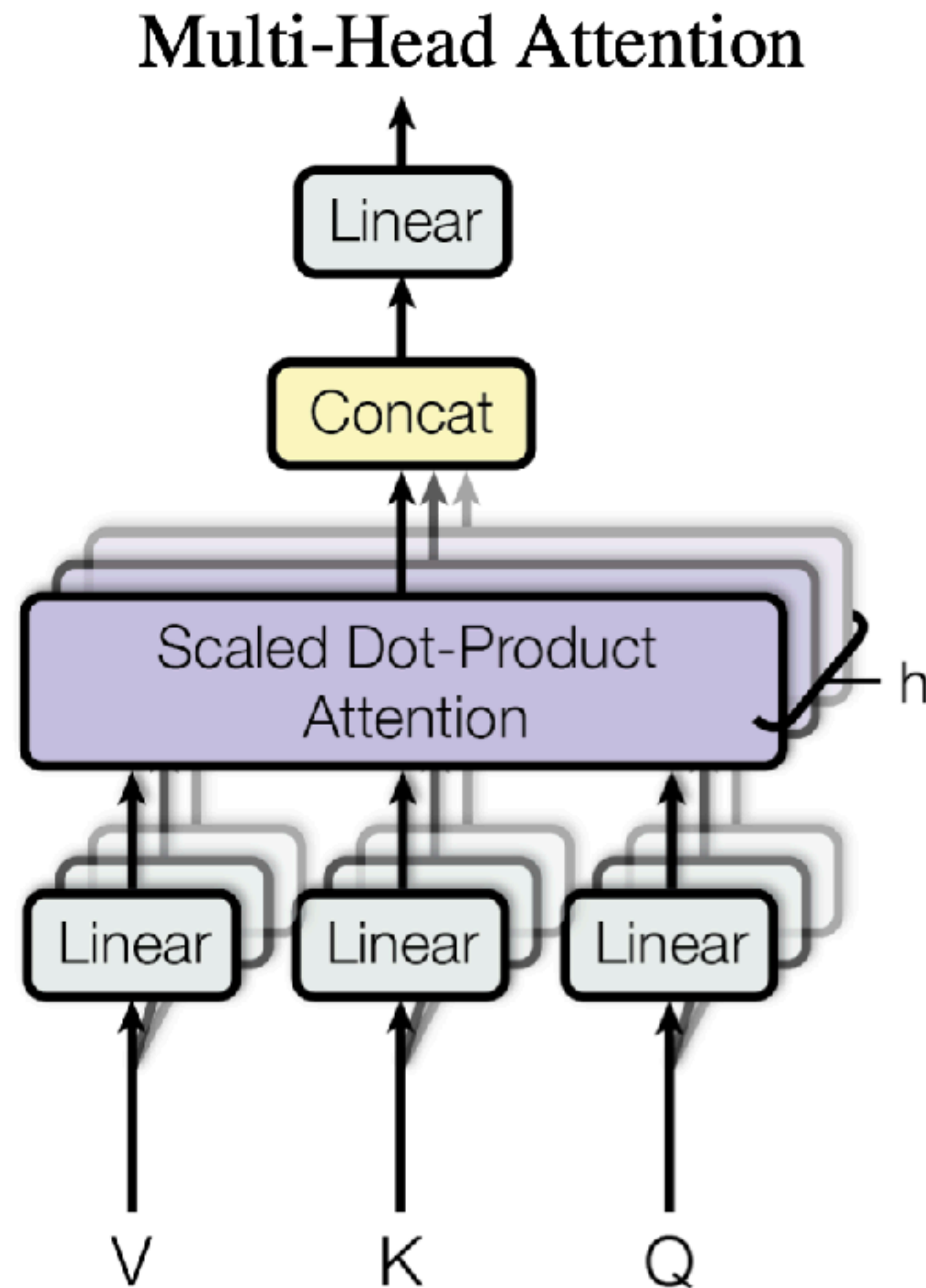
		A	brown	cat	lazes	on	the	shelf
V ₁	A			0				
V ₂	brown			0.38				
V ₃	cat			0.48				
V ₄	lazes			0.08				
V ₅	on			0.01				
V ₆	the			0				
V ₇	shelf			0.05				

Self Attention

Scaled Dot-Product Attention



Multi Head Attention



$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^O$$

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

Multi Head Attention

1) This is our input sentence*

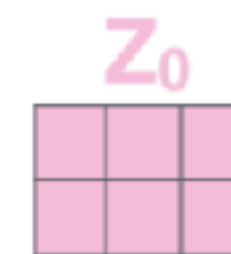
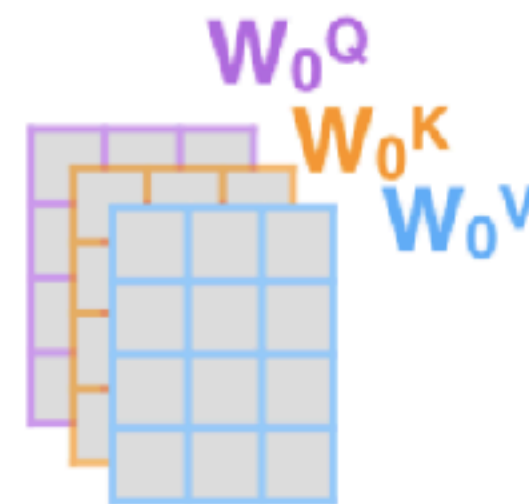
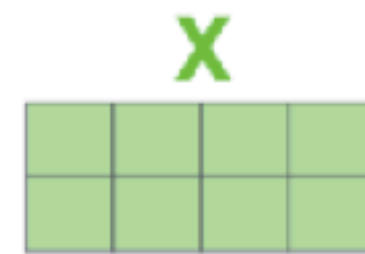
2) We embed each word*

3) Split into 8 heads. We multiply X or R with weight matrices

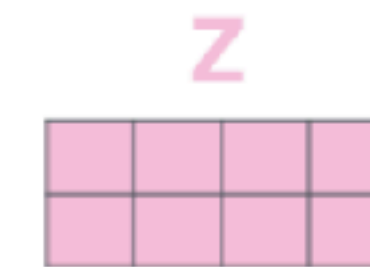
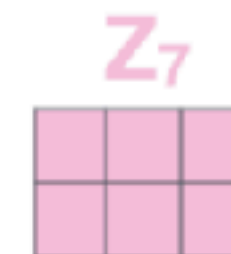
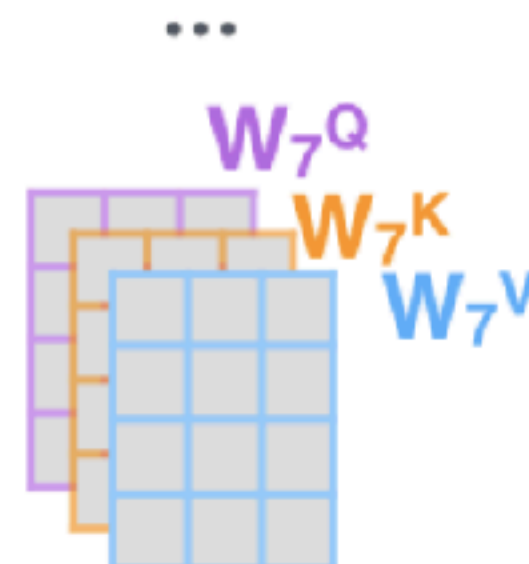
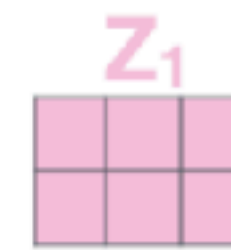
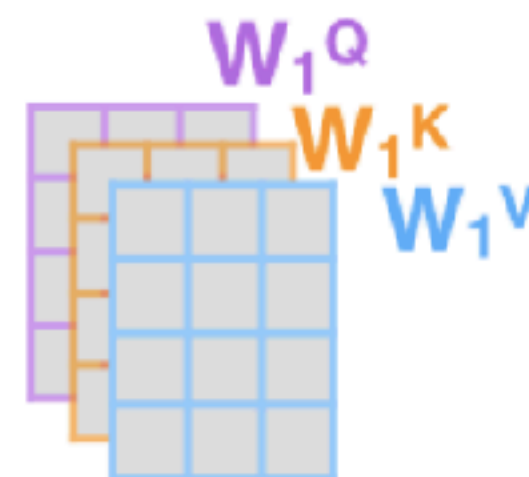
4) Calculate attention using the resulting $Q/K/V$ matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

Thinking
Machines



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



Self Attention

Gains

- Computational complexity per layer
 - Faster than RNN when seq len, $n < d$, representation dimensionality
- Parallelization
- Long distance dependency



Self Attention

Missing

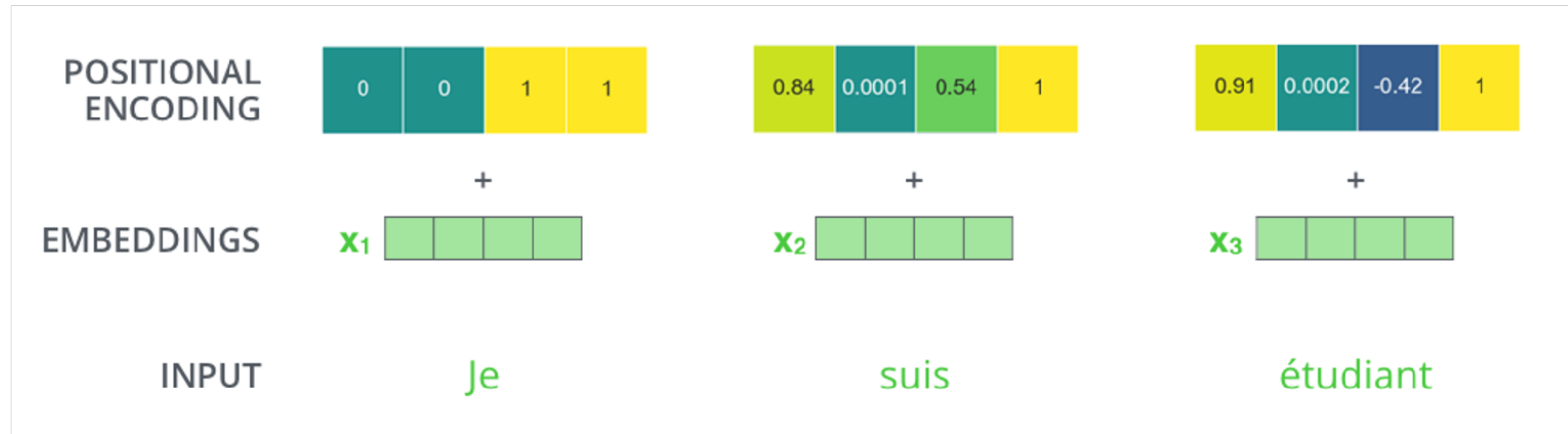
- No Word Position
- No Non-linearity
- Looks at future words



Self Attention

Positional Encoding

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$



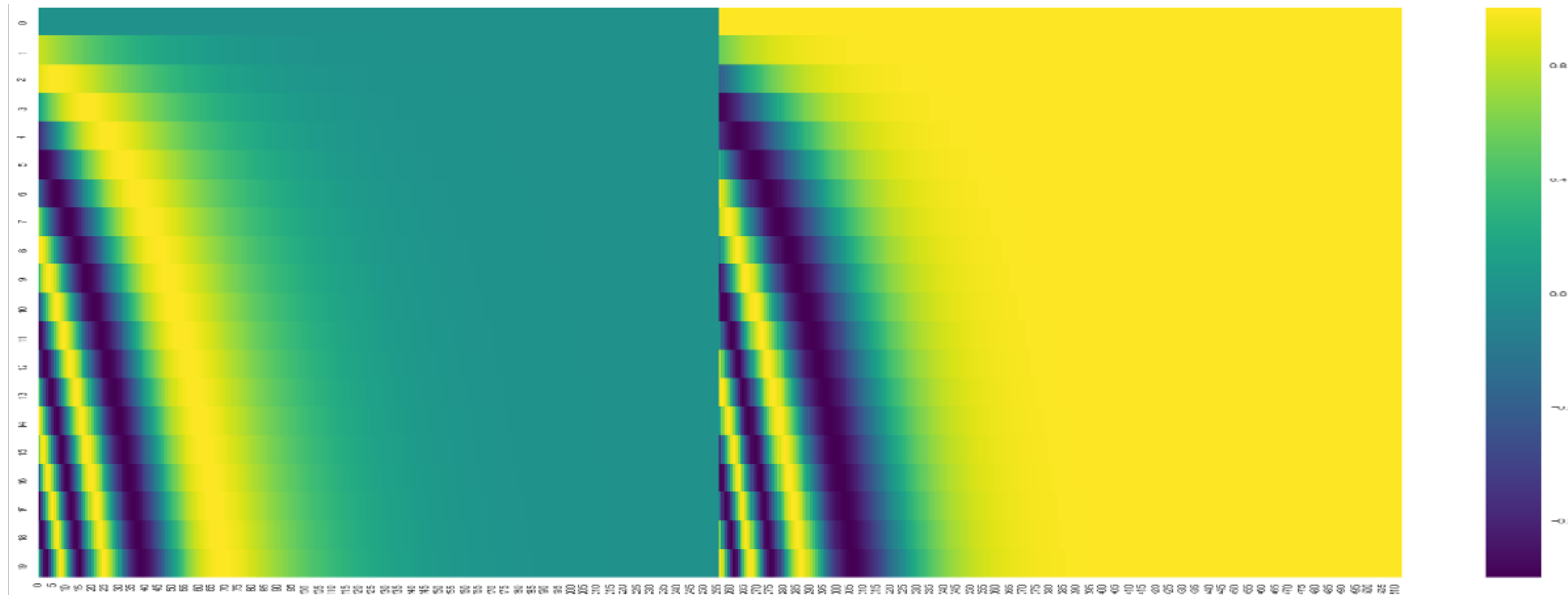
Self Attention

Positional Encoding

Sinusoidal positional encoding may allow the model to extrapolate to sequence lengths longer than the ones encountered during training

Self Attention

Positional Encoding



A real example of positional encoding for 20 words (rows) with an embedding size of 512 (columns). You can see that it appears split in half down the center. That's because the values of the left half are generated by one function (which uses sine), and the right half is generated by another function (which uses cosine). They're then concatenated to form each of the positional encoding vectors.

Self Attention

Non Linearity

Feed forward neural network to process every layer



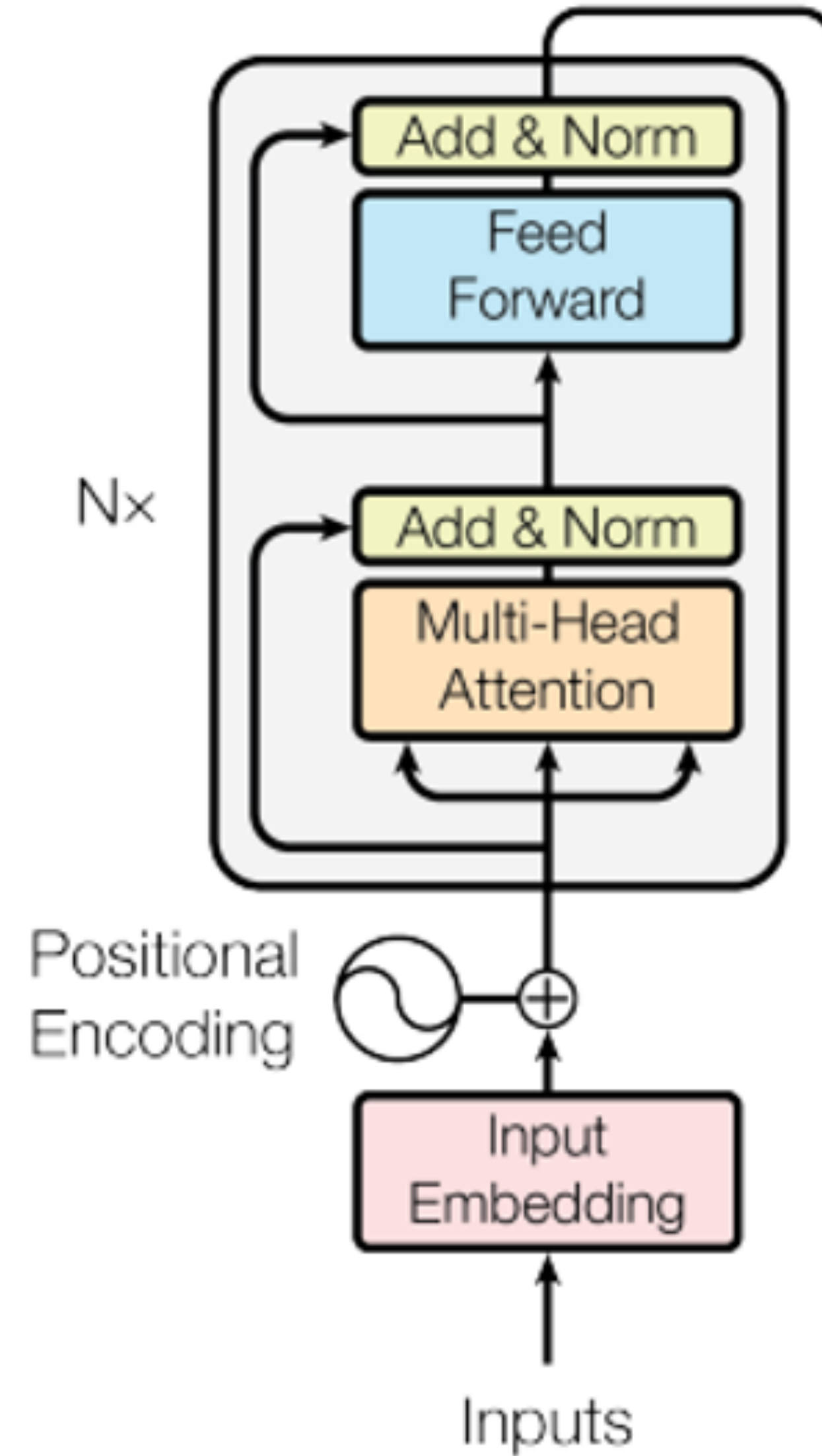
Self Attention Masking

Decoder - Causal Mask

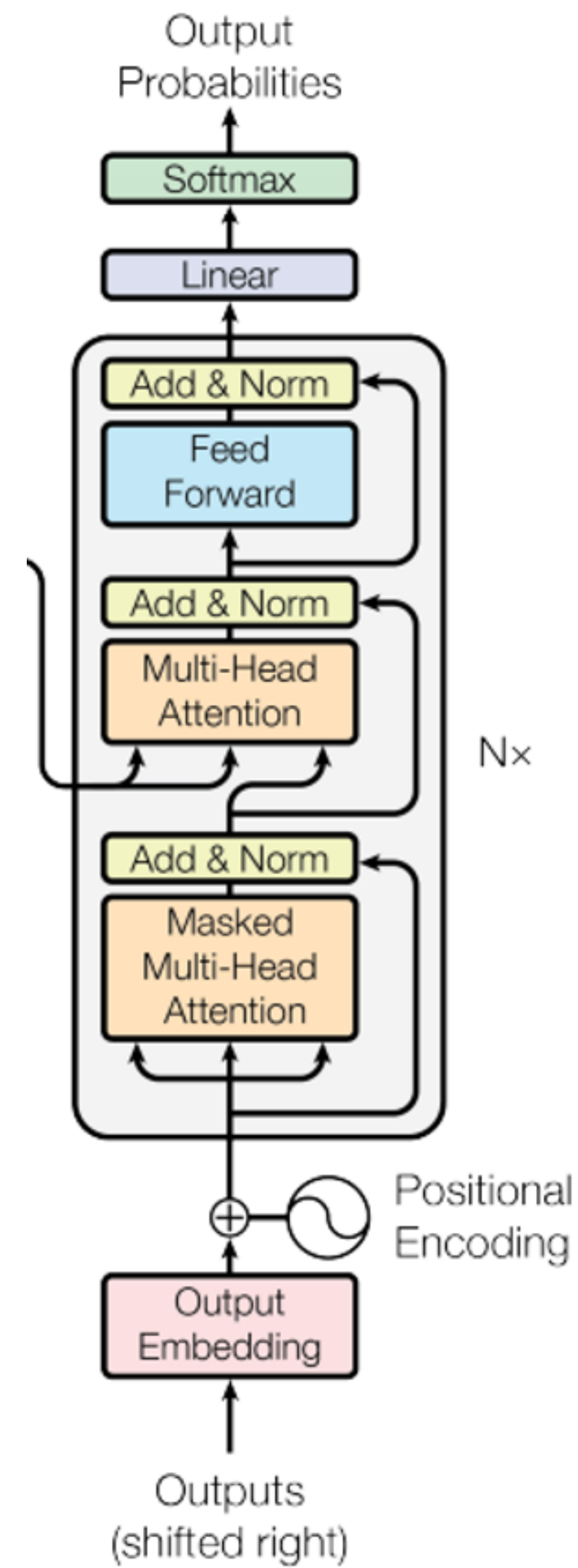
if future token set to $-\infty$ before softmax



Transformer Encoder



Transformer Decoder



Transformer Architecture

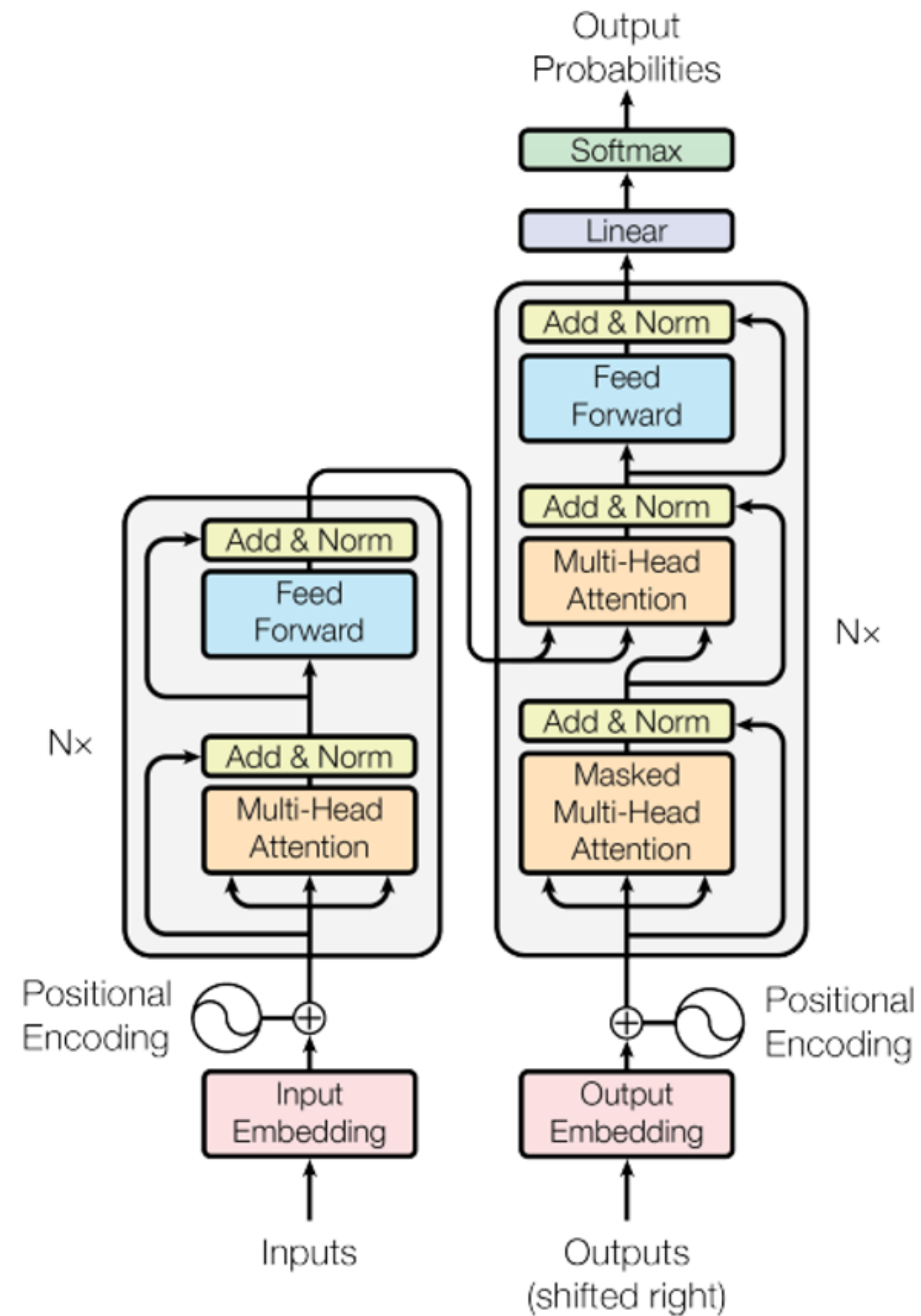


Figure 1: The Transformer - model architecture.

Add and Layer Normalization

- Residual Connections
 - Train better
 - Smooths loss function
- Layer Norm
 - Normalize to unit mean and s.d. in each layer
 - Cuts down uninformative variation in hidden vectors



Residual Connection

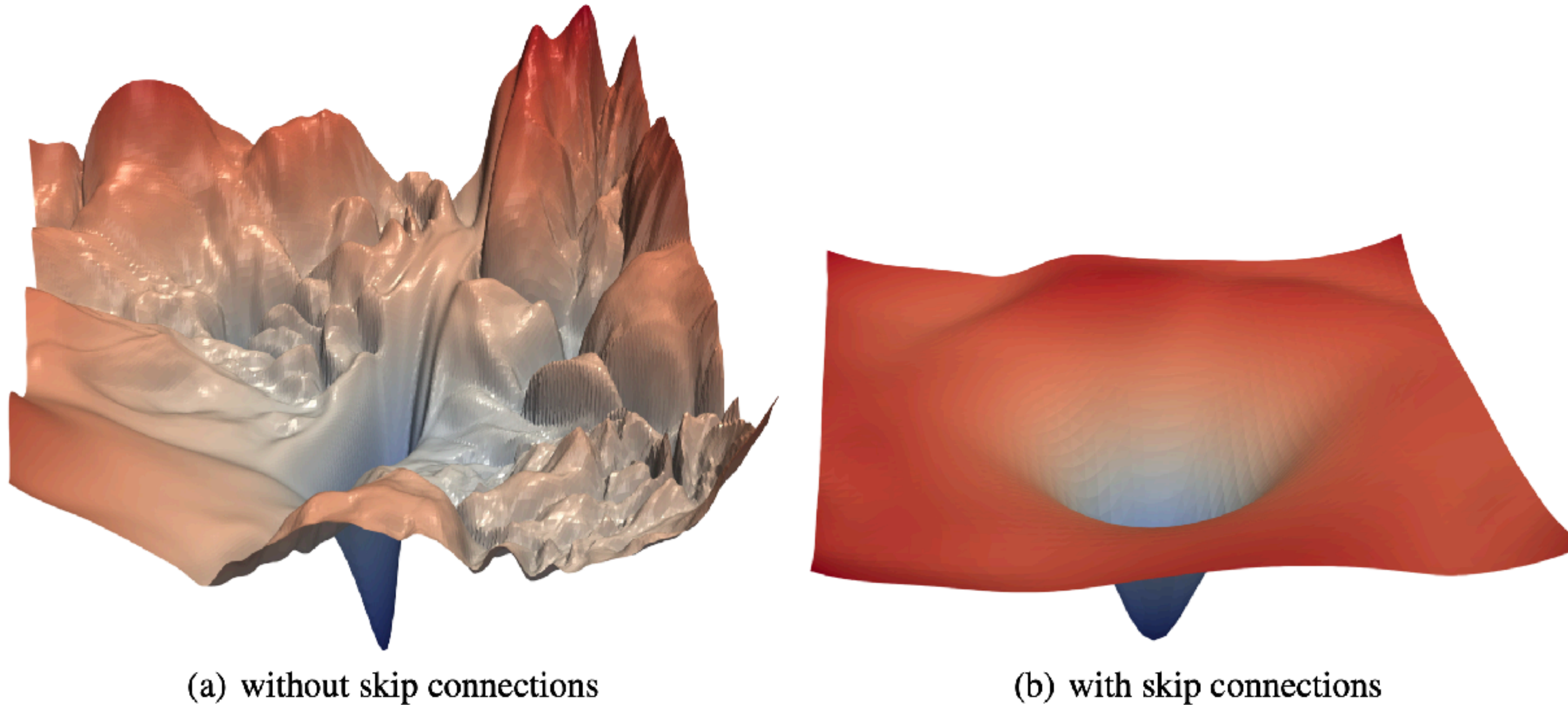
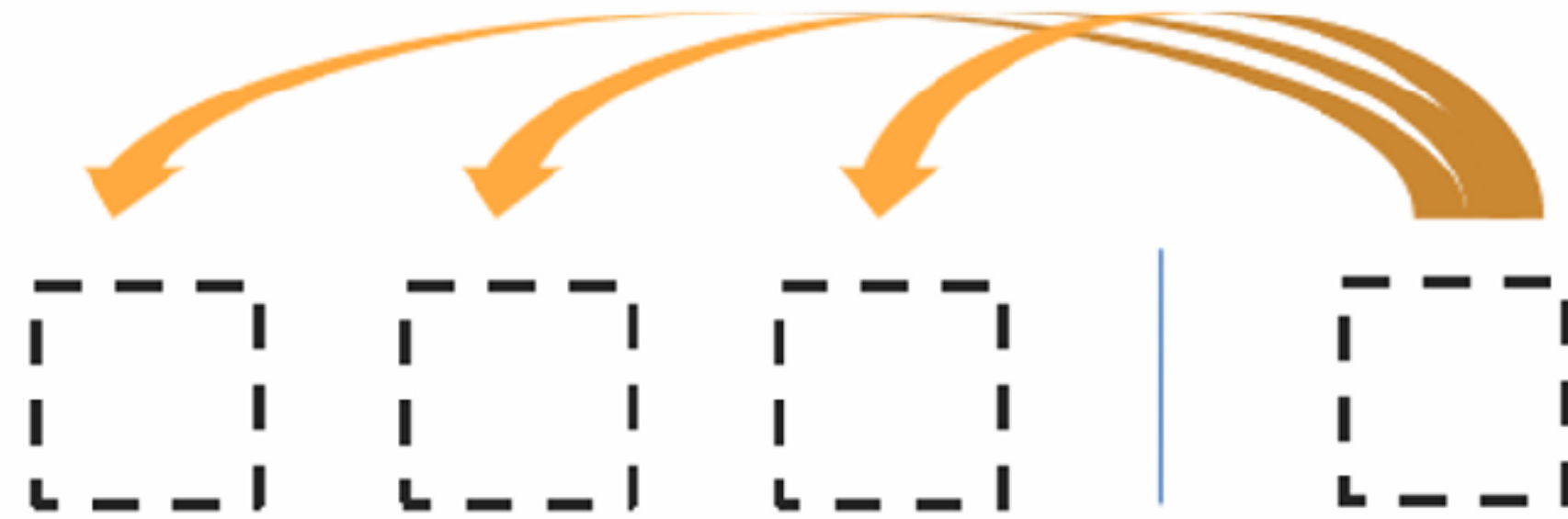


Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.

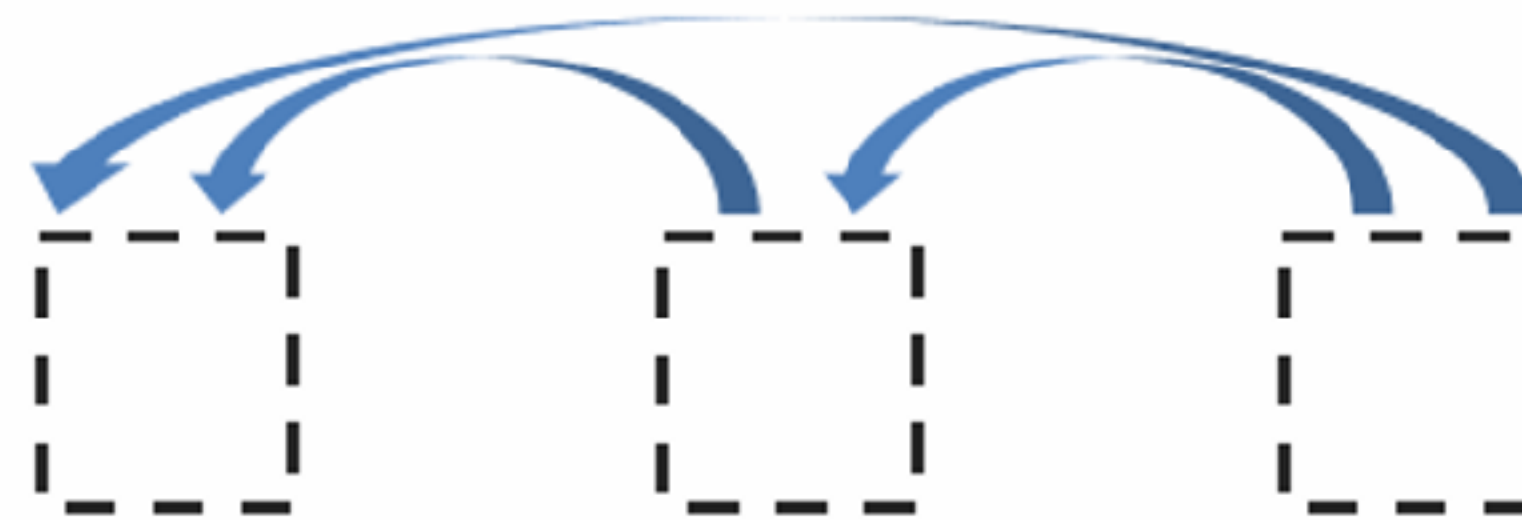
Types of Attention



Encoder-Decoder Attention



Encoder Self-Attention



Masked Decoder Self-Attention



Cross Attention

- Encoder-Decoder Attention
 - Decoder Vectors as Q
 - Encoder Output as K and V
- Every word in decoder looks for possible words from encoder
- Two distinct types of data
 - Machine Translation
 - Speech to Text



Transformers

Gains

- Parallel Processing
- Semi-supervised learning
- Transfer learning
- Knowledge-stores
- Reasoning



Transformers

Drawbacks

- Quadratic computation in Self Attention
- Position Representation



Transformers

Advances

- Position Representation
 - Learned Representations
 - Relative Position Representations
 - Absolute Position
 - Rotary Position Embeddings (RoPE)
 - NoPE



Transformers

Advances

- Quadratic computation in Self Attention
 - Sparse Attention
 - Grouped Query Attention
 - Multi Head Latent Attention
 - LogLinear Attention
 - Linear Attention

